nouez 0.7.0

Architecture and message reference

October 14, 2026

Abstract

nouez is an MCP server that lets Claude Code consult live Codex sessions. This document describes nouez 0.7.0: how the pieces fit together, and an example of every message nouez sends or acts on at each boundary: the MCP traffic between Claude Code and nouez, the WebSocket JSON-RPC traffic between nouez and the Codex app-server daemon, and the commands nouez runs on the operating system. The daemon also sends notifications and its own requests, which nouez discards and this document leaves out. Message shapes are as captured in runs with Claude Code 2.1.295 and Codex app-server daemon 0.162.0; client behaviour described here (the discovery probe, progress tokens, the 120-second move to the background) was observed in those runs and is not something nouez controls. Ids are invented, long objects are trimmed to the fields nouez reads, and quoted error texts marked as excerpts are shortened.

Contents

1 Overview **3**

2 Inside the server **3**

3 Claude Code ↔ nouez (MCP) **4**

3.1 Startup 4

3.2 Tool calls and results 6

3.3 Cancellation 7

3.4 Protocol errors 7

4 nouez ↔ Codex daemon **8**

4.1 Connecting 8

4.2 Frames 9

4.3 Message format 9

4.4 Data shapes 9

4.5 Request reference 10

4.6 Finding a session 10

5 Tools, end to end **11**

5.1 StartConsultant 11

5.2 ListConsultants 12

5.3 SendConsultantMessage 13

5.4 GetConsultantReply 15

5.5 The waiter command 16

5.6 WatchConsultant 16

5.7 StopConsultant 17

6 nouez → operating system **18**

6.1 Opening a pane 18

6.2 Detecting and closing panes 18

7 Limits and constants **18**

1 Overview

Two coding agents look at a problem differently. nouez gives each a fixed role: **Claude Code** is the main developer and owns the work; **Codex** is a consultant that Claude asks for second opinions. Consultants that nouez starts run in a read-only sandbox, and `StartConsultant` only ever reuses those. Claude can also message a Codex session the user opened, by name and with `allow_user_session: true`, when the user asks; such a session keeps the user's own permissions. Everything happens from the single Claude Code terminal.

A message makes three hops (Figure 1):

1. **Claude Code ↔ nouez.** Claude Code starts `server.py` as a child process and speaks MCP to it: JSON-RPC 2.0, one JSON object per line on stdin/stdout.
2. **nouez ↔ Codex daemon.** A tool call that needs the daemon starts `codex app-server proxy` (usually once per call; WatchConsultant may connect twice), which relays its stdio to the daemon's local control socket. Over that pipe nouez performs a WebSocket handshake and then exchanges JSON-RPC messages in WebSocket text frames.
3. **nouez → operating system.** To show a consultant to the user, nouez opens a terminal pane running `codex resume <id>`, which attaches to the same daemon session. It also checks for such panes and, on StopConsultant, tries to close them.

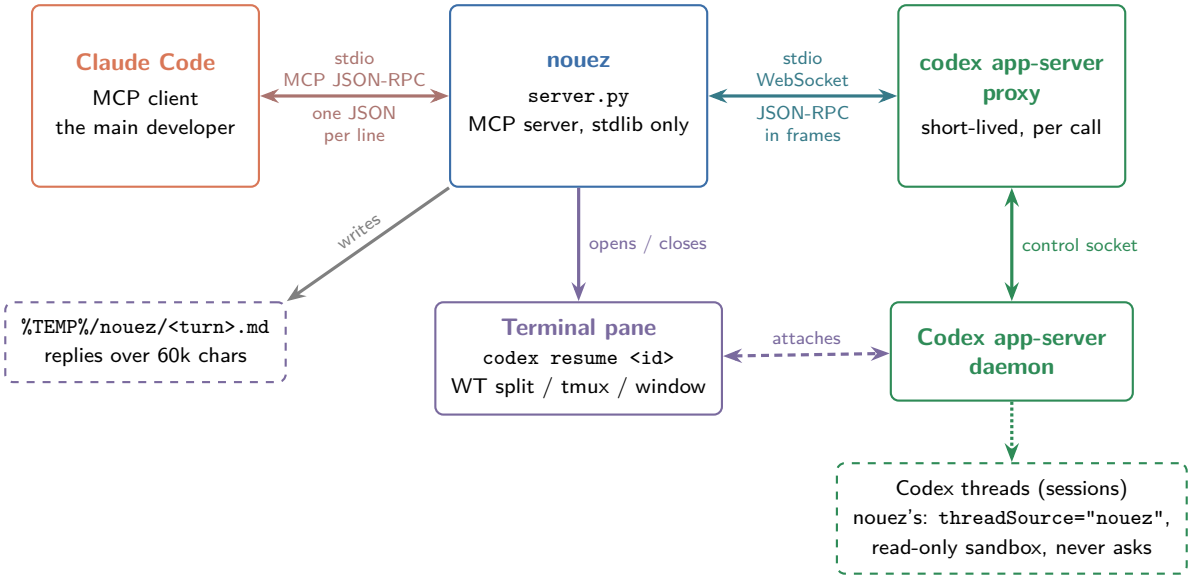


Figure 1: Components and links. Solid arrows carry messages; the terminal pane and nouez share a Codex session through the daemon, not through each other.

	Claude Code ↔ nouez	**nouez ↔ Codex daemon**	**nouez → OS**
Lifetime	whole Claude Code session	short-lived, opened by the tool call	per command
Transport	stdin/stdout of `server.py`	stdio of `codex app-server proxy`	child processes
Framing	newline-delimited JSON	WebSocket frames (RFC 6455)	argv / exit code / stdout
Protocol	MCP (JSON-RPC 2.0)	Codex app-server JSON-RPC (no `"jsonrpc"` field)	`wt`, `tmux`, `osascript`, PowerShell, `pgrep`
Who asks	mostly Claude Code; nouez asks `roots/list`	nouez (daemon requests are discarded)	always nouez
Section	3	4	6

Table 1: The three interfaces.

2 Inside the server

`server.py` is a single file using only the Python standard library. One thread reads stdin; every request runs on its own worker thread, because a tool call can block for minutes while Codex works.

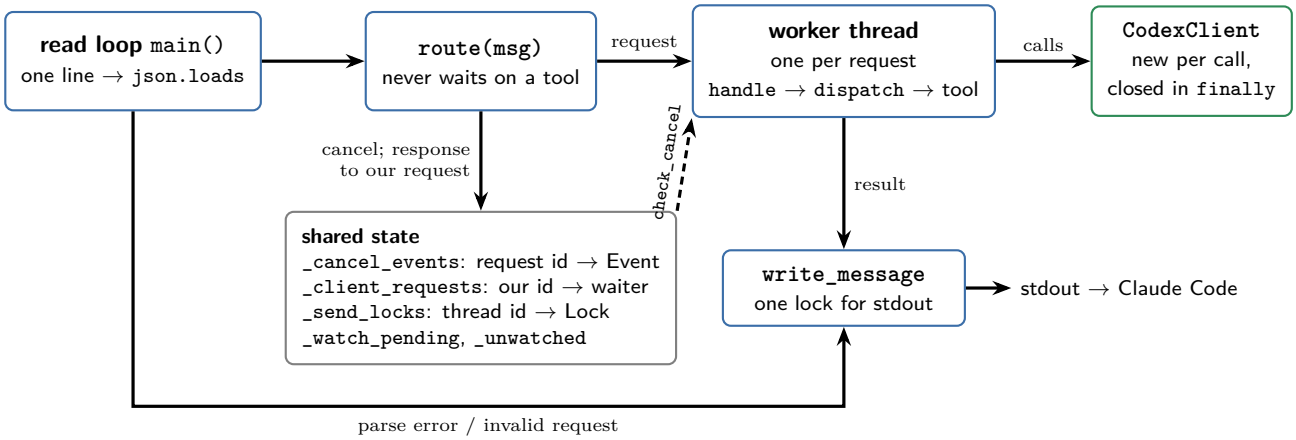


Figure 2: Threading model. Only `write_message` touches stdout, and logs go to stderr, so stdout carries nothing but MCP messages.

`route()` sorts each parsed line:

- Not an object, an `id` that isn't a string or number (booleans included), or a request with `"id": null`: answer `-32600 Invalid Request` right away, from the read loop's thread.
- No `method`: it is Claude Code's answer to a request nouez sent (`roots/list`); wake the waiting thread. Answers nobody is waiting for are dropped.
- `notifications/cancelled`: set the Event of the named request, if it is still running. Malformed params are ignored.
- Anything else: register a cancel Event for `tools/call` before starting its worker thread, so a cancel that arrives while the worker starts up is not lost. (A cancel that arrives before its request has nothing to cancel and is ignored.)

Any exception inside `route()` is logged and the loop continues, so no single line can stop the server. Inside the worker, `handle()` makes sure every request with an id gets a response unless it was cancelled: if `dispatch()` raises, it answers `-32603 Internal error: <exception>`. An exception inside a tool is different: it becomes an `isError` result reading *"Unexpected error: <exception>"*.

3 Claude Code ↔ nouez (MCP)

3.1 Startup

Claude Code 2.1.295 first probes the 2026-07-28 protocol with `server/discover`. nouez is a legacy-lifecycle server, so it answers `-32601`, and Claude Code falls back to the `initialize` handshake with protocol 2025-11-25.

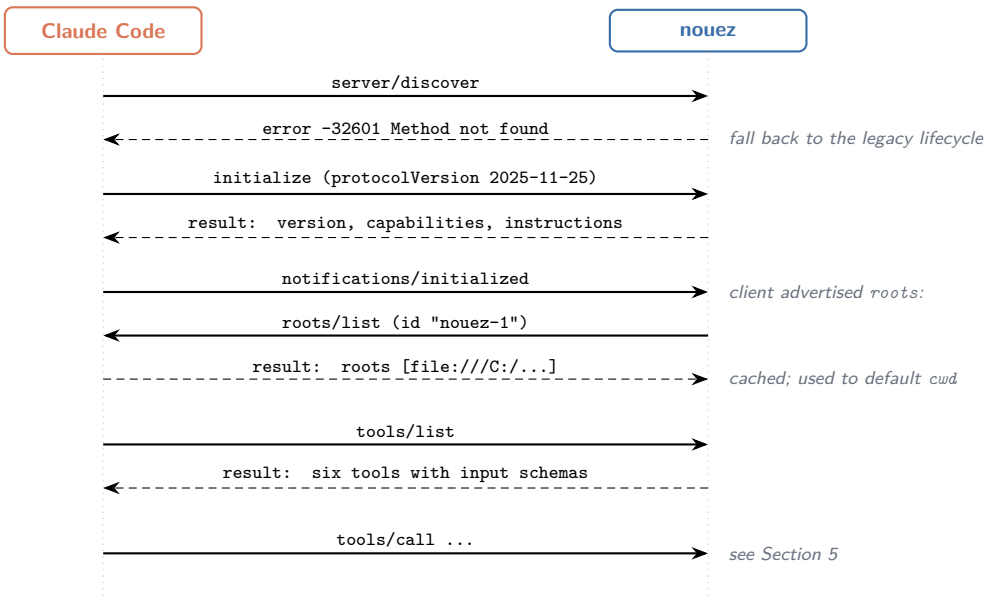


Figure 3: MCP startup.

Claude Code → nouez: discovery probe

```
{"jsonrpc": "2.0", "id": 0, "method": "server/discover", "params": {}}
```

nouez → Claude Code

```
{"jsonrpc": "2.0", "id": 0, "error": {"code": -32601, "message": "Method not found: server/discover"}}
```

Claude Code → nouez: initialize

```
{"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {
  "protocolVersion": "2025-11-25",
  "capabilities": {"roots": {"listChanged": true}, "elicitation": {"form": {}, "url": {}}},
  "clientInfo": {"name": "claude-code", "version": "2.1.295"}}}
```

nouez answers with the requested version if it supports it (2025-11-25, 2025-06-18 or 2025-03-26) and with 2025-11-25 otherwise. It never echoes an unknown version.

nouez → Claude Code

```
{"jsonrpc": "2.0", "id": 1, "result": {
  "protocolVersion": "2025-11-25",
  "capabilities": {"tools": {}},
  "serverInfo": {"name": "nouez", "version": "0.7.0"},
  "instructions": "Whenever the user asks for a Codex consultant, call StartConsultant with cwd set to your working directory,
     even if you don't remember one being open (earlier context may have been compacted). It reuses a consultant it started
     earlier in this repo, ... A result that says 'still working' is not a reply, even if it shows output so far: never treat it
     as done."}}
```

Claude Code → nouez

```
{"jsonrpc": "2.0", "method": "notifications/initialized"}
```

Because Claude Code advertised `roots`, nouez asks for its workspace folders. It does this on a worker thread and waits at most 5 seconds; a client that never answers counts as having no roots. The same request is sent again on `notifications/roots/list_changed`.

nouez → Claude Code: server-initiated request

```
{"jsonrpc": "2.0", "id": "nouez-1", "method": "roots/list", "params": {}}
```

Claude Code → nouez

```
{"jsonrpc": "2.0", "id": "nouez-1", "result": {"roots": [
  {"uri": "file:///C:/Users/you/Projects/nouez", "name": "nouez"}]}}
```

Claude Code → nouez

```
{"jsonrpc": "2.0", "id": 2, "method": "tools/list", "params": {}}
```

nouez → Claude Code (one of six tools shown)

```
{"jsonrpc": "2.0", "id": 2, "result": {"tools": [
  {"name": "SendConsultantMessage",
   "description": "Send a message into a Codex session. By default it waits 5 seconds, enough to confirm delivery and return a
     quick reply; ...",
   "inputSchema": {"type": "object",
     "properties": {
       "to": {"type": "string", "description": "Consultant name, thread id, or unique id prefix/suffix."},
       "message": {"type": "string", "description": "The message to send."},
       "from": {"type": "string", "description": "Sender label shown to Codex, ..."},
       "wait": {"anyOf": [{"type": "number", "minimum": 0, "maximum": 14400},
                          {"type": "string", "enum": ["done"]}],
                "description": "How long to wait for the reply: a number of seconds (0 = don't wait), or \"done\" ... Default 5.
     ..."},
       "allow_user_session": {"type": "boolean", "description": "Allow sending to a Codex session the user opened ... Default
     false. ..."}},
     "required": ["to", "message"]},
   "annotations": {"readOnlyHint": false, "destructiveHint": false}},
  ...]}}
```

Each tool carries annotations, hints a client may use when deciding what to confirm:

Tool	readOnlyHint	destructiveHint	also
StartConsultant	false	false	
ListConsultants	true		openWorldHint: false
SendConsultantMessage	false	false	
GetConsultantReply	true*		openWorldHint: false
WatchConsultant	false	false	idempotentHint: true, openWorldHint: false
StopConsultant	false	true	openWorldHint: false

*Its only write is nouez's own copy of an over-long reply in the temp folder (Section 5.3).

`ping` is answered with an empty result: `{"jsonrpc": "2.0", "id": 9, "result": {}}`.

3.2 Tool calls and results

In the captured runs, every `tools/call` from Claude Code 2.1.295 carried a progress token in `_meta`. Claude Code moved any call that ran past 120 seconds to the background, where its result arrives later as a task notification.

While SendConsultantMessage or GetConsultantReply waits for a running turn, nouez answers a progress token with `notifications/progress`: at most one every 10 seconds, with `progress` counting up from 1 and no `total`, since a turn's length isn't known. The message names the turn, the time waited so far and the latest line of output. None is sent once the call is cancelled, and none without a token. In the runs before 0.4.0, Claude Code did not display progress notifications.

nouez → Claude Code: progress while waiting

```
{"jsonrpc": "2.0", "method": "notifications/progress", "params": {"progressToken": 14, "progress": 3,
  "message": "codex-nouez-4d9f17 is working on turn 01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31 (waited 21s): Reading server.py ..."}}
```

Claude Code → nouez: request envelope

```
{"jsonrpc": "2.0", "id": 14, "method": "tools/call", "params": {
  "name": "SendConsultantMessage",
  "arguments": {"to": "codex-nouez-4d9f17", "message": "Review this diff for bugs: ..."},
  "_meta": {"progressToken": 14}}}
```

A tool returns text. Problems the model can act on (a bad argument, an unknown consultant, a daemon that isn't running) come back as a normal result with `isError: true`, so the model reads the message. Requests that are malformed in the ways listed in Section 3.4 get a JSON-RPC error instead.

nouez → Claude Code: success

```
{"jsonrpc": "2.0", "id": 14, "result": {"content": [{"type": "text",
  "text": "Reply from codex-nouez-4d9f17:\n\nTwo issues. 1) ..."}], "isError": false}}
```

nouez → Claude Code: tool error

```
{"jsonrpc": "2.0", "id": 15, "result": {"content": [{"type": "text",
  "text": "Codex app-server daemon is not reachable, and starting it did not help. Run `codex app-server daemon start` in a
     terminal to see why, or open a Codex session."}],
  "isError": true}}
```

3.3 Cancellation

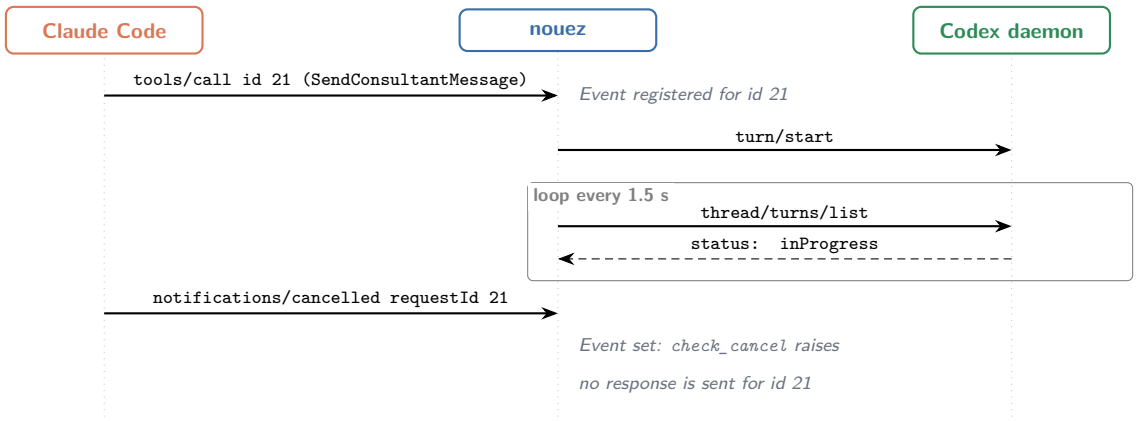


Figure 4: Cancelling a tool call. Cancelling stops nouez from waiting; it does not stop a Codex turn that has already started. Before delivery, the last `check_cancel` runs inside the send lock, so a cancel that arrives in time prevents the message from being sent at all.

Claude Code → nouez

```
{"jsonrpc": "2.0", "method": "notifications/cancelled", "params": {"requestId": 21, "reason": "User interrupted"}}
```

3.4 Protocol errors

Each of these except `-32603` is covered by `tests/test_protocol.py`; the `-32603` message is an example, since nothing in the tests makes `dispatch()` fail. After every one of them the server keeps answering.

Line that isn't JSON: `{not json` → -32700

```
{"jsonrpc": "2.0", "id": null, "error": {"code": -32700, "message": "Parse error"}}
```

Not an object (`[]`, `"hello"`), an id that isn't a string or number, or `"id": null` → -32600

```
{"jsonrpc": "2.0", "id": null, "error": {"code": -32600, "message": "Invalid Request"}}
```

Unknown method → -32601

```
{"jsonrpc": "2.0", "id": 5, "error": {"code": -32601, "message": "Method not found: nope/nope"}}
```

`"params"` that isn't an object, `[]` included → -32602

```
{"jsonrpc": "2.0", "id": 2, "error": {"code": -32602, "message": "params must be an object."}}
```

`"arguments"` that isn't an object → -32602

```
{"jsonrpc": "2.0", "id": 3, "error": {"code": -32602, "message": "arguments must be an object."}}
```

`"name": "Nope"` → -32602

```
{"jsonrpc": "2.0", "id": 4, "error": {"code": -32602, "message": "Unknown tool: Nope"}}
```

Unexpected failure while handling a request → -32603

```
{"jsonrpc": "2.0", "id": 7, "error": {"code": -32603, "message": "Internal error: KeyError('id')"}}
```

Argument values are checked by the tool and reported as `isError` results:

Tool-level validation messages

```
‘to‘ is required: the consultant name StartConsultant returned, or its thread id.
‘message‘ is required.
‘new‘ must be true or false, not ’yes’.
‘wait‘ must be a number of seconds, 0 or more (0 = don’t wait), or "done", not -5.
‘until_done‘ was replaced by ‘wait‘ in nouez 0.4: pass wait="done".
‘timeout_seconds‘ was replaced by ‘wait‘ in nouez 0.4: pass wait=<seconds>.
codex-nouez-77c2e0 is a Codex session the user opened, not a read-only consultant: it keeps the user’s own permissions and may be
     able to edit files. Nothing was sent. Only if the user asked you to message this session, send again with allow_user_session
     =true; otherwise use StartConsultant.
```

All of these are checked before anything is sent; the last one after the session is found.

4 nouez ↔ Codex daemon

4.1 Connecting

A tool call that gets past argument checks opens a fresh `CodexClient` (WatchConsultant may open a second one) and closes it in a `finally` block, killing the proxy's process tree. The connection has four steps:

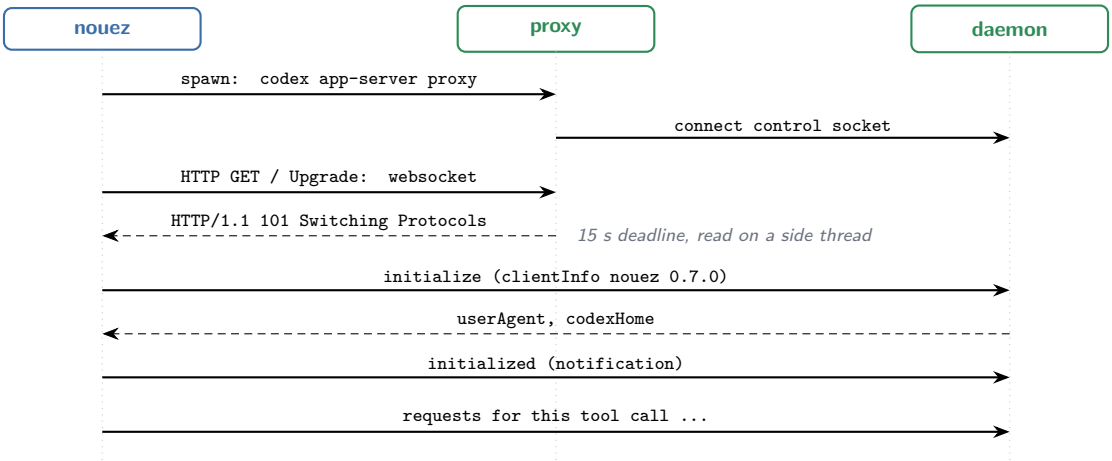


Figure 5: Opening a connection to the daemon. Bytes pass through the proxy unchanged.

nouez → proxy stdin: WebSocket upgrade

```
GET / HTTP/1.1
Host: localhost
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: q3Xv0m5cJ0n9aB1kL2pR8w==
Sec-WebSocket-Version: 13
```

proxy stdout → nouez (only the status line is checked)

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: ...
```

If the proxy closes its output before answering, no daemon is listening (it does not survive a restart of the computer). nouez then runs `codex app-server daemon start` once, without pipes and waiting at most 30 seconds for the command, and retries the connection every half second for up to 20 seconds. Only if that fails does it report *"Codex app-server daemon is not reachable, and starting it did not help. Run ‘codex app-server daemon start‘ in a terminal to see why, or open a Codex session."* If the proxy says nothing within 15 seconds, *"Codex app-server proxy did not respond within 15s."*; for any status other than 101, *"WebSocket upgrade refused: <first 200 bytes of the header>"*.

4.2 Frames

Opcode	Meaning	nouez does
`0x1` text, `0x0` continuation	a JSON message, possibly split	reassemble until FIN, then parse
`0x9` ping	keep-alive from the daemon	answer with `0xA` pong, same payload
`0x8` close	daemon is closing	stop reading; pending requests fail

Frames nouez sends are masked, as RFC 6455 requires of clients: text frames for its messages and pong frames for the daemon's pings. A received message over 64 MiB ends the connection. A reader thread puts parsed messages on a queue; `request()` sends one message and waits up to 15 seconds for the response with the same `id`. Everything else the daemon sends meanwhile, notifications and the daemon's own requests alike, is discarded. A consultant never waits on those requests, because it was started with `approvalPolicy: "never"`.

4.3 Message format

The daemon speaks JSON-RPC without the `"jsonrpc"` member. Ids are integers counting from 1 on each connection.

nouez → daemon

```
{"id": 1, "method": "initialize", "params": {"clientInfo": {"name": "nouez", "version": "0.7.0"}}}
```

daemon → nouez

```
{"id": 1, "result": {
  "userAgent": "codex-tui/0.162.0 (Windows 10.0.26200; x86_64) WindowsTerminal (nouez; 0.7.0)",
  "codexHome": "C:\\Users\\you\\.codex",
  "platformFamily": "windows", "platformOs": "windows"}}
```

nouez → daemon

```
{"method": "initialized"}
```

daemon → nouez: an error response

```
{"id": 7, "error": {"code": -32600, "message": "thread 01a1f3c2-... is not materialized yet"}}
```

nouez turns an error response into *"<method> failed: <message>"*. A few messages mean "this session has no history yet" and are treated as an empty turn list rather than a failure: `not materialized`, `missing source rollout`, `no rollout found`, `is empty`, `list_turns is not supported yet`.

4.4 Data shapes

A thread (session), fields nouez reads; the daemon sends about 30

```
{"id": "01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17",
 "cwd": "C:\\Users\\you\\Projects\\nouez",
 "model": "gpt-5.6-sol", "modelProvider": "openai",
 "status": {"type": "idle"},             // "idle" | "active" (+ "activeFlags") | "notLoaded" | "systemError"
 "threadSource": "nouez",                // null for sessions the user opened
 "ephemeral": false, "parentThreadId": null,
 "name": "Diff review", "updatedAt": 1791547200}
```

The consultant's name is derived, not stored: `codex-` + the last folder of `cwd` + `-` + the last six characters of the id, lower-cased: `codex-nouez-4d9f17`.

A turn, fields nouez reads

```
{"id": "01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31",
 "status": "completed",                  // "inProgress" | "completed" | "interrupted" | "failed"
 "error": null,                          // may be {"message": "..."} when failed or interrupted
 "items": [
   {"type": "userMessage", "content": [{"type": "text",
     "text": "[Message from Claude Code via the consultants bridge]\n\nReview this diff ..."}]},
   {"type": "agentMessage", "phase": "commentary", "text": "Reading server.py ..."},
   {"type": "agentMessage", "phase": "final_answer", "text": "Two issues. 1) ..."}]}
```

`phase` is `"commentary"` for progress messages, `"final_answer"` for the answer, or `null`. For a turn that has ended, the reply is the text of all `agentMessage` items with `phase: "final_answer"`, or, if there are none, of

the last `agentMessage`. For a turn still running, nouez shows the *output so far* instead: every `agentMessage` after the turn's latest `userMessage`, whatever its phase, keeping the last 4,000 characters. A steered message adds a `userMessage` to the running turn, so output written before it arrived is not shown as if it answered it. This is the snapshot from the latest poll; nouez polls instead of following streamed events, so it may lag the session by one poll.

4.5 Request reference

Method	Params nouez sends	Result nouez reads	Used by
`initialize`	`clientInfo`	`userAgent`, `codexHome`	every connection
`thread/loaded/list`	`{}` or `{cursor}`	`data`: ids, `nextCursor`	finding sessions
`thread/read`	`threadId`	`thread`	finding sessions
`thread/list`	`limit: 50`	`data`: summaries	finding unloaded consultants
`thread/start`	`cwd`, `sandbox`, `approvalPolicy`, `threadSource`, opt. `model`, `developerInstructions`	`thread`	StartConsultant
`thread/name/set`	`threadId`, `name`	–	StartConsultant `title`
`thread/resume`	`threadId`, `excludeTurns: true`	–	SendConsultantMessage
`thread/turns/list`*	`threadId`, `limit`, `sortDirection: "desc"`	`data`: turns, with items	send, wait, reply, watch
`turn/start`	`threadId`, `input`	`turn.id`	send to an idle session
`turn/steer`	`threadId`, `expectedTurnId`, `input`	`turnId` (not read)	send to a busy session
`thread/archive`	`threadId`	–	StopConsultant

*Codex marks `thread/turns/list` as experimental. nouez sends no `itemsView`; the daemons it was tested with returned each turn's items, but that default is not documented.

4.6 Finding a session

Every tool that takes `to` first lists sessions, then matches. Listing costs one request per session, because `thread/list` omits `threadSource`:

1. Sessions loaded in the daemon (paged)

```
-> {"id": 2, "method": "thread/loaded/list", "params": {}}
<- {"id": 2, "result": {"data": ["01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17", "01a1e8a0-..."], "nextCursor": null}}
```

2. Read each loaded session

```
-> {"id": 3, "method": "thread/read", "params": {"threadId": "01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17"}}
<- {"id": 3, "result": {"thread": {"id": "01a1f3c2-...", "cwd": "C:\\Users\\you\\Projects\\nouez", "threadSource": "nouez", "
     status": {"type": "idle"}, ...}}}
```

3. One page of up to 50 saved sessions, to find consultants the daemon has unloaded

```
-> {"id": 5, "method": "thread/list", "params": {"limit": 50}}
<- {"id": 5, "result": {"data": [{"id": "01a1d9b7-...", "status": {"type": "notLoaded"}, ...}, ...], "nextCursor": "..."}}
-> {"id": 6, "method": "thread/read", "params": {"threadId": "01a1d9b7-..."}}     // kept only if threadSource == "nouez"
```

Only that one page is read, so a consultant unloaded and older than the 50 sessions it holds is not listed. Then `to` is matched, case-insensitively: an exact name or id first; otherwise a unique id prefix or suffix (except for `StopConsultant`, which accepts only exact matches); otherwise a direct `thread/read` of `to` as a full id, which reaches older consultants. A name keeps only the id's last six characters, so two sessions can share one; such a name is refused, and the full id has to be used.

Matching failures (isError results)

```
'codex-nouez-4d9f17' names more than one session. Use the full thread id: 01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17, 01a20b55-0c9e-7a13
     -b6f1-77e0aa4d9f17
'01a1' is ambiguous. Matches: codex-nouez-4d9f17, codex-nouez-77c2e0
No Codex session named 'codex-nouez-4d9f1'. Sessions: codex-nouez-4d9f17, codex-api-0b13aa
No Codex session named '4d9f17'. Use its exact name or full thread id. Sessions: codex-nouez-4d9f17
```

5 Tools, end to end

Each diagram starts with a `tools/call` and ends with its result. "Connect" stands for the handshake in Section 4, and "find session" for the lookup in Section 4.6. The example consultant is `codex-nouez-4d9f17`.

5.1 StartConsultant

Gets the consultant for a project. It reuses the most recently updated session that nouez started in the project's git repository (outside git, in the Claude Code workspace folder that contains `cwd`), and starts a new read-only one if there is none, or if `new: true`. It never reuses a session the user opened, because such a session keeps the user's sandbox and approval policy; those are only listed in the result.

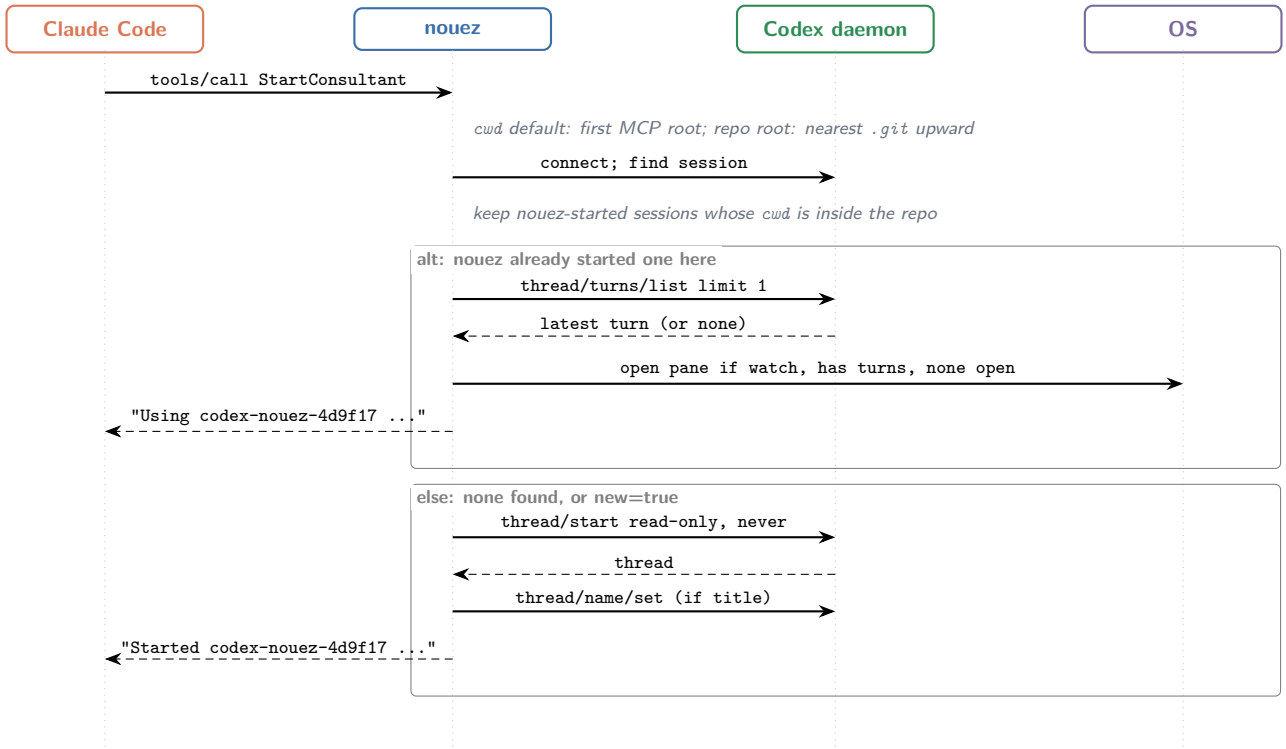


Figure 6: StartConsultant. A new session's pane opens with its first message, because `codex resume` can't show a session that has no turns yet.

Request

```
{"jsonrpc": "2.0", "id": 10, "method": "tools/call", "params": {"name": "StartConsultant",
  "arguments": {"cwd": "C:\\Users\\you\\Projects\\nouez", "title": "Diff review",
                "instructions": "You are a skeptical senior reviewer. Look for bugs; don't rewrite code."}}}
```

nouez → daemon when none exists

```
{"id": 8, "method": "thread/start", "params": {
  "cwd": "C:\\Users\\you\\Projects\\nouez", "sandbox": "read-only", "approvalPolicy": "never",
  "threadSource": "nouez",
  "developerInstructions": "You are a skeptical senior reviewer. Look for bugs; don't rewrite code."}}
{"id": 9, "method": "thread/name/set", "params": {"threadId": "01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17", "name": "Diff review"}}
```

Result text: started

```
Started codex-nouez-4d9f17
    id: 01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17
    cwd: C:\Users\you\Projects\nouez
    model: gpt-5.6-sol (openai)
    sandbox: read-only
A pane showing it opens beside Claude Code when you send its first message.
Send it work with SendConsultantMessage.
```

Result text: reused

```
Using codex-nouez-4d9f17, which already works in C:\Users\you\Projects\nouez; nothing new was started.
- codex-nouez-4d9f17
    id: 01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17
    cwd: C:\Users\you\Projects\nouez
    model: gpt-5.6-sol (openai)
    status: idle (unloaded; reloads on the next message)
    started by: StartConsultant
    title: Diff review
Opened codex-nouez-4d9f17 in a Windows Terminal split pane.
Ignored title, instructions: they apply only to a new session (new=true).
Send it work with SendConsultantMessage.
The user's own Codex sessions in this repo were not used (they keep the user's permissions); message one only if the user asks you
      to:
- codex-nouez-77c2e0
    id: 01a1e8a0-1c3d-7f02-8e44-90b1d577c2e0
    ...
    started by: user
Only if the user asks for a separate consultant, call StartConsultant with new=true.
```

The pane line is omitted with `watch: false`. If the session has no turns yet it reads *"A pane showing it opens beside Claude Code when you send its first message."*, and if a terminal already shows it, nothing is opened and the line is *"... is already open in a terminal; no new one was opened."* The `watch` value of the latest StartConsultant call also decides whether later sends open a pane. The "Ignored" line appears only when `model`, `title` or `instructions` were passed. Other nouez sessions in the same repository follow *"Other consultants in this repo:"*, and the user-session note appears only when there are user sessions; it is added to the "Started" result too.

Both results end with an update notice when the `codex` CLI on the PATH or the daemon is older than the latest Codex release. nouez takes the latest version from `<codexHome>/version.json`, the cache Codex keeps for its own update check, so it makes no network request. The CLI version comes from `codex -version` and the daemon's from `userAgent`. The notice names only the part that is behind and appears once per server process. It doesn't appear when the cache is missing or the user dismissed that version in Codex. nouez updates nothing itself; Claude runs the commands if the user asks:

Result text: update notice

```
Codex 0.165.0 is out; the codex CLI on PATH is 0.163.0 and the app-server daemon is 0.162.1. Tell the user and go ahead with the
     consultation. Only if the user asks you to update, run these in a shell: ‘codex update‘ updates the CLI; ‘codex app-server
     daemon update‘ updates the daemon, which may interrupt running Codex work, consultants and the user's own sessions included.
```

5.2 ListConsultants

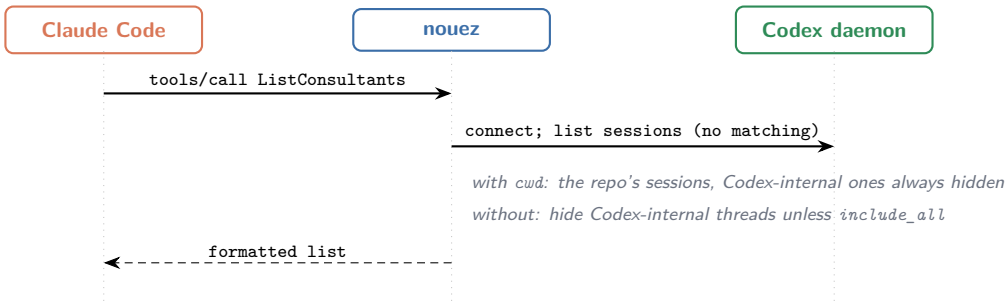


Figure 7: ListConsultants. Read-only; makes no changes. It lists the user's sessions too.

Request

```
{"jsonrpc": "2.0", "id": 11, "method": "tools/call", "params": {"name": "ListConsultants",
  "arguments": {"cwd": "C:\\Users\\you\\Projects\\nouez"}}}
```

Result text

```
2 Codex session(s) in C:\Users\you\Projects\nouez:
- codex-nouez-4d9f17
    id: 01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17
    cwd: C:\Users\you\Projects\nouez
    model: gpt-5.6-sol (openai)
    status: busy
    started by: StartConsultant
- codex-nouez-77c2e0
    id: 01a1e8a0-1c3d-7f02-8e44-90b1d577c2e0
    cwd: C:\Users\you\Projects\nouez
    model: gpt-6-astra (openai)
    status: idle
    started by: user
```

Result text: nothing found

```
No Codex sessions in C:\Users\you\Projects\nouez. Start one with StartConsultant, or open Codex in a terminal.
```

5.3 SendConsultantMessage

The central tool. A lock per session, held only inside this nouez process, makes sure two of its concurrent sends can't both start a turn; it does not cover other clients of the daemon, such as the user typing in a Codex terminal. The lock is held through delivery, the check that the new turn is listed, and opening the pane, and is released before the long wait, so a slow reply doesn't block other senders.

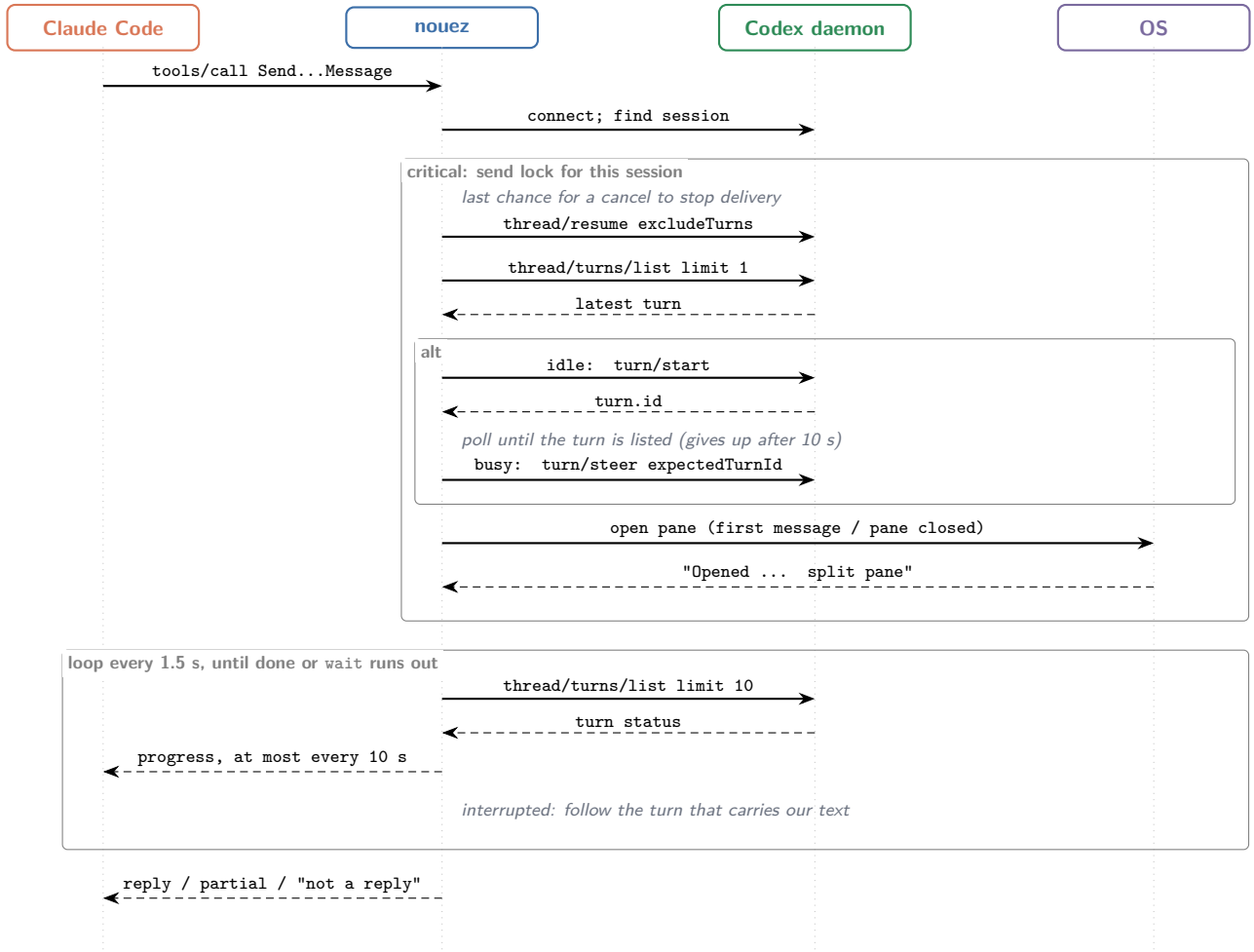


Figure 8: SendConsultantMessage with a `wait` above zero (the default is 5 seconds). Before the lock, a session the user opened is refused unless `allow_user_session: true`, and nothing is sent. Progress notifications go out only if the request carried a progress token. With `wait: 0` it returns after the steps inside the lock (delivery, the listing check and the pane), skipping only the wait for the reply. The 10 s check stops polling after 10 s, but a slow `thread/turns/list` request can take up to its own 15 s timeout, so it is not a hard bound.

Request

```
{"jsonrpc": "2.0", "id": 14, "method": "tools/call", "params": {"name": "SendConsultantMessage",
  "arguments": {"to": "codex-nouez-4d9f17", "message": "Review this diff for bugs:\n\n```diff\n...\n```",
                "from": "Claude Code (nouez)", "wait": 600}}}
```

Every message reaches Codex with a header naming the sender:

nouez → daemon: session idle

```
{"id": 4, "method": "thread/resume", "params": {"threadId": "01a1f3c2-...", "excludeTurns": true}}
{"id": 5, "method": "thread/turns/list", "params": {"threadId": "01a1f3c2-...", "limit": 1, "sortDirection": "desc"}}
{"id": 6, "method": "turn/start", "params": {"threadId": "01a1f3c2-...", "input": [{"type": "text",
  "text": "[Message from Claude Code (nouez) via the consultants bridge]\n\nReview this diff for bugs: ..."}]}}
```

daemon → nouez

```
{"id": 6, "result": {"turn": {"id": "01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31", ...}}}
```

nouez → daemon: session busy, so the message steers the running turn

```
{"id": 6, "method": "turn/steer", "params": {"threadId": "01a1f3c2-...",
  "expectedTurnId": "01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31",
  "input": [{"type": "text", "text": "[Message from Claude Code via the consultants bridge]\n\nAlso check the tests."}]}}
```

Waiting: polled every 1.5 s

```
-> {"id": 9, "method": "thread/turns/list", "params": {"threadId": "01a1f3c2-...", "limit": 10, "sortDirection": "desc"}}
<- {"id": 9, "result": {"data": [{"id": "01a1f3c4-...", "status": "inProgress", "items": [...]}, ...]}}
   ...
<- {"id": 61, "result": {"data": [{"id": "01a1f3c4-...", "status": "completed", "items": [...]}, ...]}}
```

Result text: completed

```
Reply from codex-nouez-4d9f17:

Two issues. 1) route() accepts float ids but ...

(Opened codex-nouez-4d9f17 in a Windows Terminal split pane.)
```

Result text: wait=0

```
Delivered to codex-nouez-4d9f17 (started a new turn, turn 01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31). Don't wait for the reply. To be
     woken when it is ready, run this as a background command (in Claude Code: the Bash or PowerShell tool with run_in_background
     ); it exits with the reply when the turn ends:
C:/Python313/python.exe -I C:/Users/you/nouez/server.py wait 01a1f3c2-5e10-7c3a-9f44-2b8d1e4d9f17 01a1f3c4-8b20-7d55-b1e3-6
     a9c0f2e7d31
Then tell the user the consultation is under way and carry on with other work or end your turn. If you can't run background
     commands, read the reply later with GetConsultantReply (turn_id 01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31).
```

Result text: timeout (isError false; the message was delivered)

```
codex-nouez-4d9f17 is still working on turn 01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31; this is not a reply. Delivered (started a new
     turn), but no reply within 600s. Don't send the message again. Don't wait for the reply. To be woken when it is ready, run
     this as a background command (in Claude Code: the Bash or PowerShell tool with run_in_background); it exits with the reply
     when the turn ends:
C:/Python313/python.exe -I C:/Users/you/nouez/server.py wait 01a1f3c2-5e10-7c3a-9f44-2b8d1e4d9f17 01a1f3c4-8b20-7d55-b1e3-6
     a9c0f2e7d31
Then tell the user the consultation is under way and carry on with other work or end your turn. If you can't run background
     commands, read the reply later with GetConsultantReply (turn_id 01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31).

Output so far (a snapshot of the running turn; it may change and is not the final answer):
Reading server.py ... route() looks fine; checking dispatch next.
```

The "Output so far" block is left out when the turn has written nothing since the message arrived. If an interrupted turn carried on in a new one, the turn id given is the new one's.

Result text: waiting failed after delivery (isError true)

```
Delivered to codex-nouez-4d9f17 (started a new turn, turn 01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31), but waiting for the reply failed:
      thread/turns/list timed out after 15s. Don't send the message again; read the reply with GetConsultantReply (turn_id 01
     a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31).
```

Result text: the turn ended without completing

```
codex-nouez-4d9f17: turn 01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31 interrupted.
Partial reply:
Reading server.py ...
If the message was folded into a later turn, read it with GetConsultantReply.
```

Result text: reply over 60,000 characters

```
Reply from codex-nouez-4d9f17:

<first 60,000 characters>

[Truncated: 18422 more characters. The full reply is saved in C:\Users\you\AppData\Local\Temp\nouez\01a1f3c4-8b20-7d55-b1e3-6
    a9c0f2e7d31.md.]
```

5.4 GetConsultantReply

Reads a reply without sending anything. A send returns after 5 seconds by default, usually before the reply is ready, and Claude is told not to wait for it but to run the waiter (next section) in the background. Without background commands, it calls this tool with the returned `turn_id` when the user asks or when it next needs the reply, and the call returns at once. When the user does want Claude to wait, it passes `wait: "done"`. That call returns when the turn ends, when an error occurs (such as a lost daemon connection), or after the 4-hour cap, so a client that moved it to the background is notified when the reply is ready, up to one 1.5-second poll late. An error or the cap is not a reply; call again.

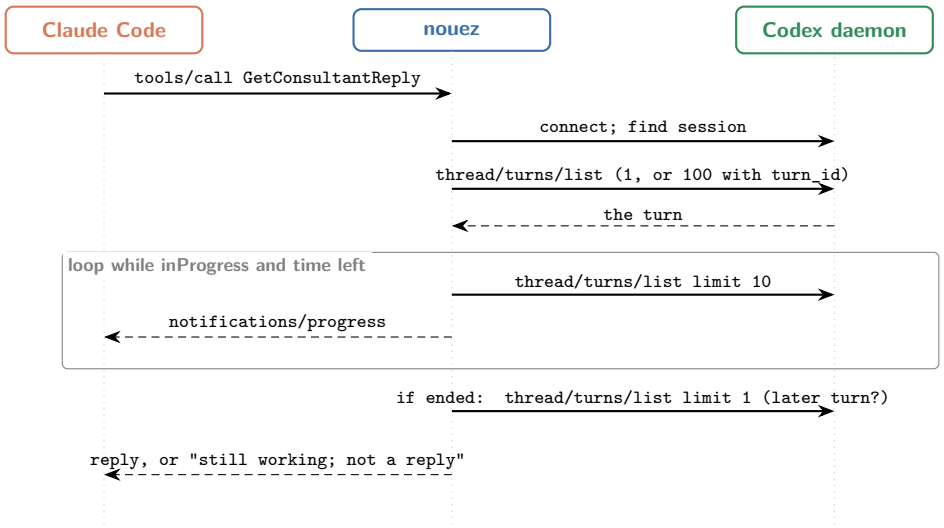


Figure 9: GetConsultantReply. `wait` defaults to 0: return at once. It reads user sessions too; only sending is guarded.

Request

```
{"jsonrpc": "2.0", "id": 16, "method": "tools/call", "params": {"name": "GetConsultantReply",
  "arguments": {"to": "codex-nouez-4d9f17", "turn_id": "01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31", "wait": "done"}}}
```

Result text: variants

```
Reply from codex-nouez-4d9f17:

Two issues. 1) ...

codex-nouez-4d9f17 is still working on turn 01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31; this is not a reply. To be woken when it is
    ready, run in the background:
C:/Python313/python.exe -I C:/Users/you/nouez/server.py wait 01a1f3c2-5e10-7c3a-9f44-2b8d1e4d9f17 01a1f3c4-8b20-7d55-b1e3-6
    a9c0f2e7d31
Or check again later with turn_id 01a1f3c4-8b20-7d55-b1e3-6a9c0f2e7d31; pass wait="done" only if the user wants you to wait for it
    .

Output so far (a snapshot of the running turn; it may change and is not the final answer):
Reading server.py ...

codex-nouez-4d9f17 has no messages yet.

codex-nouez-4d9f17 has no turn 01a1ffff-... among its 100 most recent turns.
```

An interrupted turn followed by a newer turn adds: *"A later turn exists (01a1f3c6-..., completed). If your message continued there, read it by omitting turn_id."* nouez does not check that the later turn holds the message here; the newer turn may be someone else's.

5.5 The waiter command

`server.py wait <thread_id> <turn_id>` is not a tool but a command line. A send that returns before the reply, and a `GetConsultantReply` that finds the turn still working, give it with full paths (`sys.executable -I` and the absolute path of `server.py`) and tell Claude to run it as a background command. In Claude Code that is the Bash or PowerShell tool with `run_in_background`: Claude is free at once, and when the command exits Claude Code wakes it with the command's output, so the reply arrives without the user asking for it. A path is quoted only if it contains a space (the command then adds that PowerShell needs `&` before it), so the command runs as is in either shell and a permission rule can match it by prefix.

`server.py allow-rule` prints those rules for the interpreter that runs it, one per shell tool:

```
Add these to permissions.allow in ~/.claude/settings.json:
"Bash(C:/Python313/python.exe -I C:/Users/you/nouez/server.py wait *)",
"PowerShell(C:/Python313/python.exe -I C:/Users/you/nouez/server.py wait *)"
```

They name one interpreter and one script, so Claude Code keeps them in auto mode, which drops only broad interpreter rules; without them it asks before each waiter, or auto mode may block it.

The waiter opens its own connection, resolves the thread by exact id, and looks for the turn among its `REPLY_LOOKBACK` (100) most recent turns. A turn that is missing ends it with exit code 1. A turn that has already ended is printed at once; a running one, which is always among the newest, is polled with `wait_for_turn` for up to `WAITER_SECONDS` (25 minutes). Claude Code stops a background command after 30 minutes by default, so at that point the waiter exits with code 0 and the *still working* result, including the output so far and the same command to run again; Claude starts a new waiter. When the turn ends, the waiter prints *"The consultant's turn has ended (thread ..., turn ...)."* and the turn formatted as `GetConsultantReply` would, with the reply cut at `WAITER_REPLY_CHARS` (20,000) characters and the rest saved to a file, and exits with code 0. Its standard output is UTF-8.

A failed connection (`BridgeError`, or an `OSError` such as a broken pipe) is retried 10 seconds later, never past the 25-minute deadline. After `WAITER_RETRIES` (3) failures in a row, a count that each working connection resets, it prints *"Stopped waiting for turn ..."* with a pointer to `GetConsultantReply` and exits with code 1. The waiter dies with the Claude Code session that started it; the reply stays readable with `GetConsultantReply`.

5.6 WatchConsultant

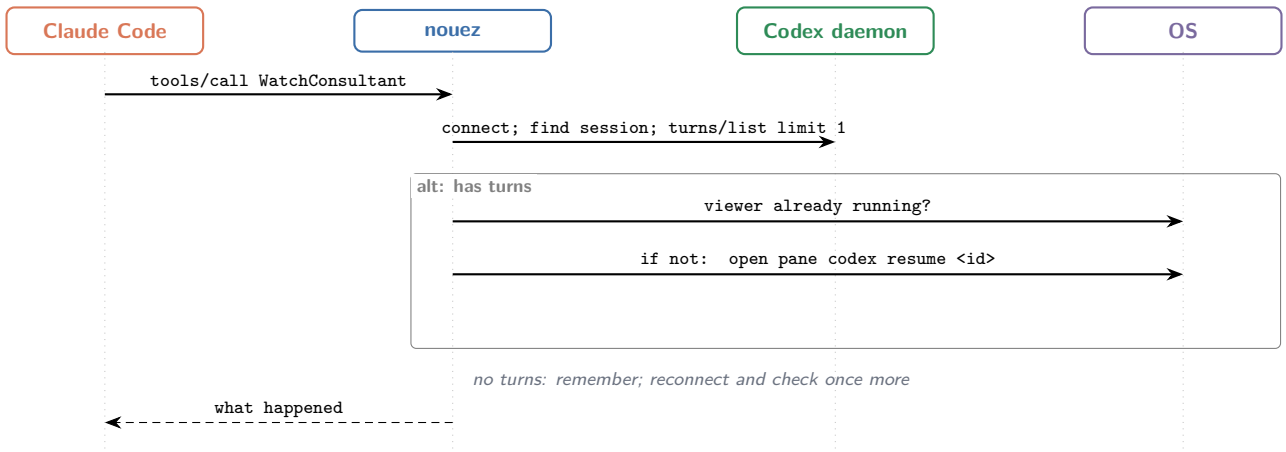


Figure 10: WatchConsultant. With no turns, it records the request and opens a second connection to check again, in case a send slipped in; otherwise the next SendConsultantMessage opens the pane. It works on the user's sessions too.

Request


```
{"jsonrpc": "2.0", "id": 17, "method": "tools/call", "params": {"name": "WatchConsultant", "arguments": {"to": "codex-nouez-4d9f17
    "}}}
```


Result text: variants

```
Opened codex-nouez-4d9f17 in a Windows Terminal split pane.
codex-nouez-4d9f17 is already open in a terminal; no new one was opened.
codex-nouez-4d9f17 has no messages yet; a terminal window will open when you send the first one.
Opened codex-nouez-4d9f17 in Windows Terminal. Warning: the codex CLI on PATH is 0.160.1 but the daemon is 0.162.0, so the window
    may ask to restart the daemon. Choose Cancel (restarting interrupts running consultants) and update the codex CLI to 0.162.0.
```

5.7 StopConsultant

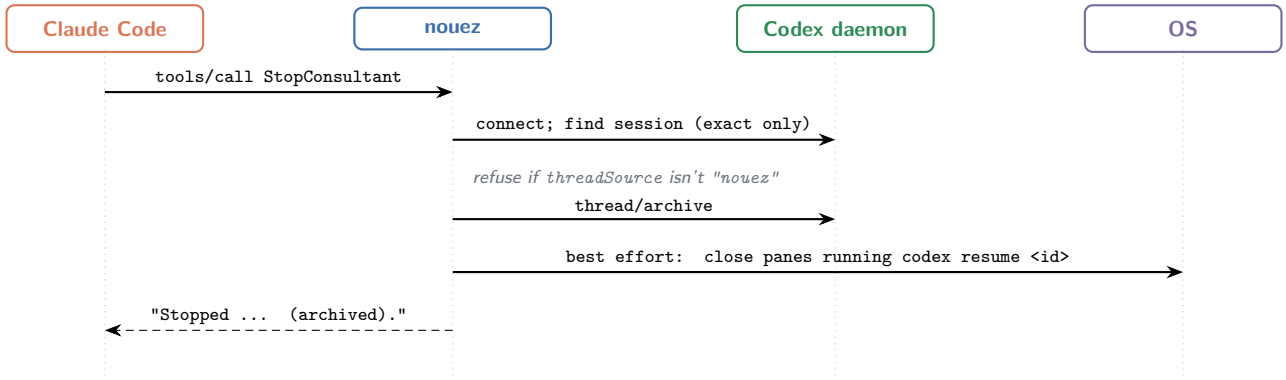


Figure 11: StopConsultant. Archiving is reversible: `codex unarchive <id>`. Panes are found by command line; the count in the result is how many matched, and a failed close is not reported.

Request


```
{"jsonrpc": "2.0", "id": 18, "method": "tools/call", "params": {"name": "StopConsultant", "arguments": {"to": "codex-nouez-4d9f17"
    }}}
```


nouez → daemon


```
{"id": 7, "method": "thread/archive", "params": {"threadId": "01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17"}}
```


Result text

```
Stopped codex-nouez-4d9f17 (archived). Closed 1 terminal window(s) showing it. To bring it back: codex unarchive 01a1f3c2-5e7b-7
    c41-9a0d-2b6e8c4d9f17, then codex resume 01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17.
```

Result text: refused (isError true)

```
codex-nouez-77c2e0 was not started by StartConsultant. Close it from its own terminal.
```

6 nouez → operating system

6.1 Opening a pane

The pane runs the daemon's own `codex` binary when nouez can find it, because a viewer whose version differs from the daemon's can refuse to attach and offer to restart the shared daemon. nouez reads the daemon's version from `userAgent` and looks under `<codexHome>/packages/app-server-daemon/releases/<version>-*/bin/`, confirming with `codex -version`.

Where Claude Code runs	Command nouez starts
tmux (`$TMUX` set)	`tmux split-window -h -c <cwd> '<codex> resume <id>'`
Windows Terminal (`$WT_SESSION`)	`wt -w 0 split-pane -V -d <cwd> -title "Codex consultant" cmd /k <codex> resume <id>`
Windows, `wt` installed	`wt -w new -d <cwd> -title "Codex consultant" cmd /k <codex> resume <id>`
Windows, no `wt`	`cmd /c start "Codex consultant" cmd /k <codex> resume <id>`
macOS	`osascript -e 'tell application "Terminal" to do script "cd <cwd> && <codex> resume <id>"' -e '... activate'`
Linux	`x-terminal-emulator -e | gnome-terminal - | xterm -e <codex> resume <id>`

6.2 Detecting and closing panes

Thread ids are checked against `[0-9A-Za-z-]+` before they appear in any command: opening a pane refuses a malformed id with an error, and the checks and closing skip it.

Is a viewer already running? (Windows)

```
powershell -NoProfile -NonInteractive -Command
  @(Get-CimInstance Win32_Process -Filter "Name='codex.exe'" |
    Where-Object { $_.CommandLine -like '* resume 01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17*' }).Count
-> 1
```

Is a viewer already running? (macOS, Linux)

```
pgrep -f "codex[^ ]* resume 01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17"
```

Closing on StopConsultant (Windows), abridged: find the cmd windows nouez opened, with their children

```
powershell ... Get-CimInstance Win32_Process -Filter "Name='cmd.exe'" |
  Where-Object { $_.CommandLine -like '*/k *codex* resume 01a1f3c2-...' } | ForEach-Object { "$p $k" }
-> 18244 20112
taskkill /PID 20112 /T /F           (codex inside the pane)
TerminateProcess(18244, exit code 0) (cmd; a clean exit lets Windows Terminal close the pane)
```

Closing on StopConsultant (macOS, Linux)

```
pkill -f "codex[^ ]* resume 01a1f3c2-5e7b-7c41-9a0d-2b6e8c4d9f17"
```

The last two lines are what nouez does, not commands it runs: `taskkill` is run per child, and the cmd process is ended through the Win32 `TerminateProcess` call.

Each check and close command has a timeout (20 s; 10 s for `codex -version`). When it fires, nouez stops reading the command's pipes, because on Windows a grandchild process can keep a pipe open forever, and kills the process: the whole tree on Windows (`taskkill /T`), only the direct process elsewhere. The pane itself is started without waiting, so it has no timeout.

7 Limits and constants

Constant	Value	Meaning
PROTOCOL_VERSIONS	2025-11-25, 2025-06-18, 2025-03-26	MCP versions answered
SANDBOX	`read-only`	sandbox of every started consultant
BRIDGE_SOURCE	`nouez`	`threadSource` tag on started sessions
POLL_SECONDS	1.5 s	turn polling interval
PROGRESS_SECONDS	10 s	least time between progress notifications
MAX_WAIT_SECONDS	4 h	cap on polling for a turn, `wait: "done"` included
MAX_REPLY_CHARS	60,000	longer replies are cut and saved to a file
PARTIAL_CHARS	4,000	output so far shown for a running turn (the end)
RECENT_SAVED_THREADS	50	saved sessions checked for unloaded consultants
REPLY_LOOKBACK	100	turns searched for a `turn_id`
MAX_FRAME_BYTES	64 MiB	largest WebSocket message accepted
Daemon request timeout	15 s	per Codex request and for the handshake
`roots/list` timeout	5 s	then nouez assumes no roots
Default `wait`	5 s / 0 s	SendConsultantMessage / GetConsultantReply
DAEMON_START_SECONDS	20 s	wait for a daemon nouez started to accept connections
WAITER_REPLY_CHARS	20,000	longest reply the waiter prints
WAITER_RETRIES	3	failed connections the waiter rides out
WAITER_SECONDS	25 min	one waiter run, under Claude Code's limit